

DOCUMENTACION TECNICA

Registro del dispositivo y backend

ElatoAI sobre ESP32-S3 Zero

REGISTRO-DISPOSITIVO-BACKEND.MD

Registro del dispositivo y backend — ElatoAI sobre ESP32-S3 Zero

TABLA DE CONTENIDOS

Registro del dispositivo y backend — ElatoAI sobre ESP32-S3 Zero	2
Descripcion general.....	2
Flujo de operacion principal	4
Arquitectura	5
Archivos involucrados	6
Funcionamiento interno.....	6
Que significa "registrar el dispositivo"	6
Flujo de autenticacion del firmware	7
Uso del token en el WebSocket	8
El "atajo" para desarrollo: saltar el registro	8
Persistencia (NVS)	8
Manual de usuario.....	9
Requisitos previos	9
Eleccion del modo (paso clave).....	9
Setup paso a paso (tu propio backend: PROD_MODE)	10
Setup rapido sin desplegar nada (ELATO_MODE)	10
Personalizacion	11
Solucion de problemas.....	12
Referencia rapida.....	13

Descripcion general

Este documento explica la pieza del proyecto **ElatoAI** que queda *fuera* del firmware del hardware: **el registro del dispositivo y la configuracion del backend / credenciales**. Es la respuesta a la frase "falta lo de siempre del proyecto: registrar el dispositivo y configurar el backend (modo `ELATO_MODE`)", pero eso es logica de servidor, ajena al hardware".

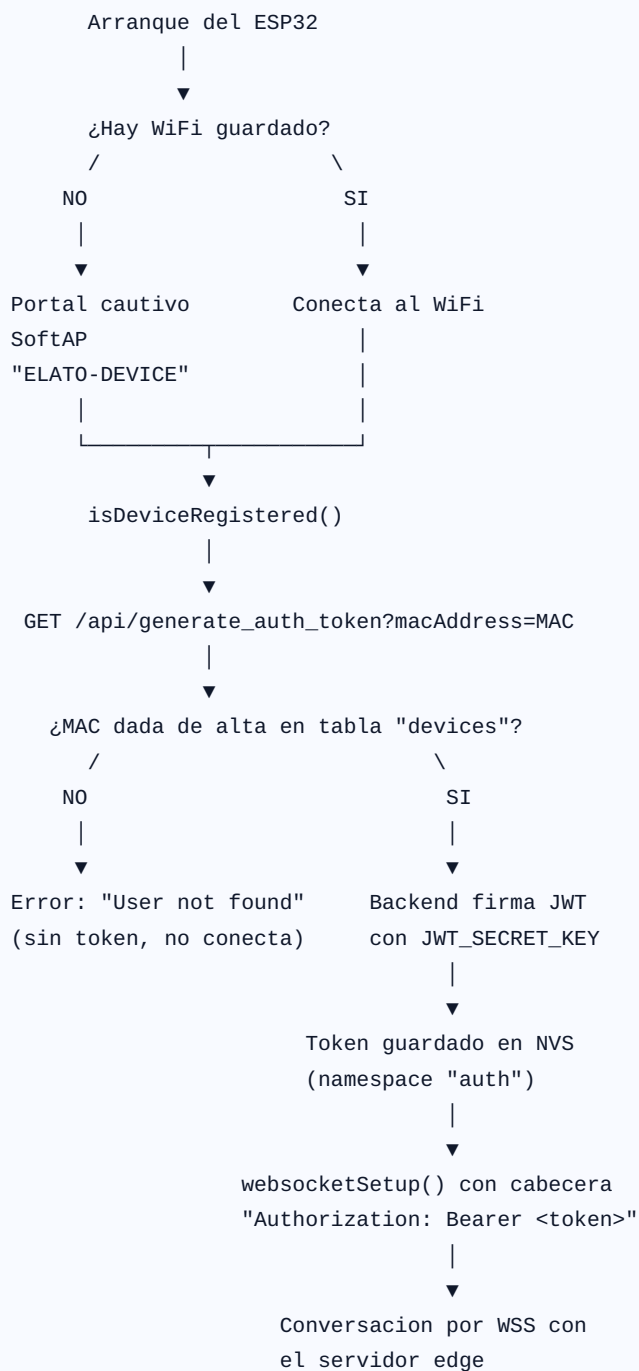
ElatoAI no es solo firmware. Es un sistema de **tres componentes** que se comunican por HTTPS y WebSockets seguros (WSS):

1. **Frontend** — aplicación web Next.js desplegada en Vercel, con base de datos **Supabase** (Postgres). Aquí creas tu cuenta, defines agentes de IA (personalidad + voz) y **das de alta tu dispositivo por su dirección MAC**.
2. **Servidor edge** — funciones **Deno Edge** o **Cloudflare Workers** que reciben el WebSocket del ESP32 y hablan con el proveedor de IA (OpenAI, Gemini, xAI, ElevenLabs, Hume...).
3. **Cliente ESP32** — tu placa (el ESP32-S3 Zero adaptado). Solo guarda *URLs de configuración* y un *token de autenticación*; no contiene ninguna clave de IA ni la personalidad del agente.

El firmware **no usa MQTT ni Bluetooth para datos**: toda la conversación viaja por WebSocket seguro hacia el servidor edge.

NOTA El ESP32 nunca almacena claves de OpenAI/Gemini ni la personalidad del agente. Solo guarda en su memoria NVS (flash) un **token JWT** que el backend le entrega tras reconocer su MAC. Por eso esta configuración es idéntica para cualquier placa: el LED WS2812, el touch o el ESP32-S3 Zero no cambian nada de este flujo.

Flujo de operacion principal



Arquitectura



Archivos involucrados

Archivo	Rol
<code>firmware-arduino/src/Config.h</code>	Selecciona el modo (<code>DEV</code> / <code>PROD</code> / <code>ELATO</code>) y el backend de voz (<code>DENO</code> / <code>CLOUDFLARE</code>). Declara URLs y certificados.
<code>firmware-arduino/src/Config.cpp</code>	Define las URLs (<code>ws_server</code> , <code>backend_server</code>), puertos, rutas WSS y los certificados CA segun el modo elegido.
<code>firmware-arduino/src/WifiManager.cpp</code>	<code>isDeviceRegistered()</code> : pide el token al backend con la MAC y lo guarda en NVS. Portal cautivo SoftAP.
<code>firmware-arduino/src/Audio.cpp</code>	<code>websocketSetup()</code> : abre el WSS con la cabecera <code>Authorization: Bearer <token></code> y <code>X-Device-Mac</code> .
<code>firmware-arduino/src/main.cpp</code>	<code>getAuthTokenFromNVS()</code> carga el token al arrancar; nombre del SoftAP <code>"ELATO-DEVICE"</code> .
<code>frontend-nextjs/app/api/generate_auth_token/route.ts</code>	Endpoint del backend que busca la MAC en <code>devices</code> , firma el JWT con <code>JWT_SECRET_KEY</code> y lo devuelve.
<code>frontend-nextjs/.env.example</code>	Variables del frontend: Supabase, <code>JWT_SECRET_KEY</code> , <code>NEXT_PUBLIC_SKIP_DEVICE_REGISTRATION</code> , claves de IA.
<code>server/deno/.env.example</code>	Variables del servidor edge Deno: Supabase, <code>JWT_SECRET_KEY</code> , claves de los proveedores de IA.
<code>server/cloudflare/.dev.vars.example</code>	Variables del Worker de Cloudflare (incluye <code>JWT_SECRET_KEY</code>).

Funcionamiento interno

Que significa "registrar el dispositivo"

"Registrar el dispositivo" **no es algo que se hace en el hardware**. Es dar de alta tu placa en la base de datos para que el backend sepa a que usuario y a que agente de IA pertenece. Segun el comentario del propio endpoint (`generate_auth_token/route.ts`):

Steps to register your device:

- 1: Register the device `mac\_address` and `user\_code` in the `devices` table.
- 2: Make sure the user adds the `user\_code` to their account in Settings to link the device to their `user\_id`.
- 3: When NEXT_PUBLIC_SKIP_DEVICE_REGISTRATION is false, we fetch the user by `mac\_address`.

En la practica, en el frontend:

1. Se inserta una fila en la tabla `devices` con la **MAC** del ESP32 y un `user_code`.
2. El usuario introduce ese `user_code` en **Settings** de la webapp, lo que asocia el dispositivo a su `user_id` (y por tanto a su agente de IA).

TIP Para saber la MAC de tu ESP32-S3 Zero, abre el monitor serie (`pio device monitor`) y reinicia: el firmware la imprime. Tambien existe el test `test/print_mac_address_test.cpp`. Recuerda: el Zero solo expone USB-C nativo, asi que el monitor funciona gracias a `ARDUINO_USB_CDC_ON_BOOT=1`.

Flujo de autenticacion del firmware

`WifiManager.cpp -> isDeviceRegistered()` es el corazon del proceso. Tras conectarse al WiFi, el dispositivo pide su token:

```
// Modo produccion/ELATO (HTTPS con certificado CA)
WiFiClientSecure client;
client.setCACert(Vercel_CA_cert);
http.begin(client, "https://" + String(backend_server) +
  "/api/generate_auth_token?macAddress=" + WiFi.macAddress());
...
String authToken = doc["token"];
preferences.begin("auth", false);
preferences.putString("auth_token", authToken); // <- persiste en NVS (flash)
```

El backend (`generate_auth_token/route.ts`) busca la MAC, obtiene el usuario y firma un JWT de Supabase (algoritmo `HS256`) con `JWT_SECRET_KEY`, **sin expiracion** (`expireDays = null`):

```
const user = await getUserByMacAddress(macAddress); // tabla "devices"
const token = createSupabaseToken(process.env.JWT_SECRET_KEY!, payload, null);
return NextResponse.json({ token });
```

Si la MAC **no** esta registrada, el endpoint responde `"User not found"` y el dispositivo se queda sin token (no podra conectar al WebSocket).

Uso del token en el WebSocket

Una vez obtenido, el token viaja en cada conexion WSS como cabecera HTTP (`Audio.cpp -> websocketSetup()`):

```
const String headers =
  "Authorization: Bearer " + String(authTokenGlobal) + "\r\n" +
  "X-Wifi-Rssi: " + String(WiFi.RSSI()) + "\r\n" +
  "X-Device-Mac: " + WiFi.macAddress();
...
websocket.beginSslWithCA(server_domain.c_str(), port, path.c_str(), CA_cert);
```

El servidor edge valida ese JWT contra el mismo `JWT_SECRET_KEY` antes de aceptar la conversacion.

ADVERTENCIA El `JWT_SECRET_KEY` debe ser **identico** en el frontend (quien firma el token) y en el servidor edge (quien lo valida). Si no coinciden, el dispositivo obtiene token pero el WebSocket lo rechaza.

El "atajo" para desarrollo: saltar el registro

Si en el frontend defines `NEXT_PUBLIC_SKIP_DEVICE_REGISTRATION=True`, el endpoint **ignora la MAC** y usa siempre un usuario de desarrollo (`admin@elatoai.com`):

```
if (skipDeviceRegistration) {
  user = await getDevUser();           // admin@elatoai.com
} else {
  user = await getUserByMacAddress(macAddress);
}
```

TIP Usa `SKIP_DEVICE_REGISTRATION=True` para probar rapido tu placa sin dar de alta la MAC. Ponlo en `False` para produccion / multiples dispositivos.

Persistencia (NVS)

El token se guarda en la particion **NVS** de la flash (namespace `"auth"`, clave `"auth_token"`). Al arrancar, `main.cpp -> getAuthTokenFromNVS()` lo carga en `authTokenGlobal`. Por eso solo se pide al backend la primera vez (o tras un *factory reset*).

NOTA La nueva tabla de particiones de 4 MB del ESP32-S3 Zero (`partition.csv`) mantiene la particion `nvs`, asi que el token sobrevive a reinicios y actualizaciones OTA igual que en la placa original de 16 MB.

Manual de usuario

Requisitos previos

- Una cuenta y proyecto en **Supabase** (o Supabase local con CLI).
- El **frontend Next.js** desplegado (Vercel) o corriendo en local.
- Un **servidor edge** desplegado: Deno Deploy o Cloudflare Workers.
- Claves de API del proveedor de IA que vayas a usar (p. ej. `OPENAI_API_KEY`).
- La **direccion MAC** de tu ESP32-S3 Zero.

NOTA Si solo quieres probar sin desplegar nada propio, puedes usar `ELATO_MODE`, que apunta a los servidores ya hospedados de Elato (`www.elatoai.com`). En ese caso el registro se hace en la web de Elato.

Eleccion del modo (paso clave)

El modo se elige **descomentando una sola linea** en `firmware-arduino/src/Config.h`:

```
// #define DEV_MODE      // servidores locales (tu PC), sin SSL
// #define PROD_MODE     // TUS servidores propios desplegados, con TUS certificados
#define ELATO_MODE      // servidores hospedados por Elato (por defecto)

// Backend de voz (independiente del modo):
// #define VOICE_SERVER_DENO
#define VOICE_SERVER_CLOUDFLARE
```

Diferencias resumidas (valores reales de `Config.cpp`):

Modo	<code>ws_server</code> / <code>backend_server</code>	Puerto WSS	SSL	Para quien
<code>DEV_MODE</code>	IP local (ej. <code>192.168.1.33</code>)	8000 (Deno) / 8787 (CF)	<code>OFF</code> (http/ws)	Desarrollo en tu PC
<code>PROD_MODE</code>	<code><tu>.deno.dev</code> / <code><tu>.workers.dev</code> + tu backend Vercel	443	<code>ON</code> (tus certs)	Tu propio despliegue
<code>ELATO_MODE</code>	<code>talkedge.deno.dev</code> / <code>elato.akash-b25.workers.dev</code> + <code>www.elatoai.com</code>	443	<code>ON</code> (certs incluidos)	Usar la nube de Elato

Backend de voz	ws_path	Notas
VOICE_SERVER_DENO	/	Funciones Deno Edge
VOICE_SERVER_CLOUDFLARE	/ws/esp32	Cloudflare Workers + Durable Objects

Setup paso a paso (tu propio backend: PROD_MODE)

1. **Supabase:** crea el proyecto y aplica el esquema (tablas `users`, `devices`, etc.). Apunta tu `SUPABASE_URL` y `ANON_KEY`.

2. **Frontend** (`frontend-nextjs/.env`): copia `.env.example` y rellena:

```
bash NEXT_PUBLIC_SUPABASE_URL=<URL de tu proyecto Supabase>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<anon key> JWT_SECRET_KEY=<una clave de 32+ caracteres>
NEXT_PUBLIC_SKIP_DEVICE_REGISTRATION=False OPENAI_API_KEY=<tu clave>
```

1. **Servidor edge** (`server/deno/.env` o `server/cloudflare/.dev.vars`): usa **el mismo** `JWT_SECRET_KEY` y las claves de IA.

2. **Registra tu dispositivo** en la webapp: da de alta la **MAC** del ESP32 (tabla `devices`) y vincula el `user_code` en *Settings*. Asocia un agente.

3. **Firmware** (`Config.h`): selecciona `PROD_MODE` y el backend de voz. En `Config.cpp`, sustituye las URLs `<your-...>` por las tuyas y pega tus certificados CA (`CA_cert` y `Vercel_CA_cert`).

4. **Compila y flashea** (recuerda el modo descarga del Zero: manten **BOOT** pulsado, conecta USB-C, suelta):

```
bash cd firmware-arduino pio run -t upload pio run -t uploadfs # sube startup.mp3 a SPIFFS pio
device monitor
```

1. **Conecta el WiFi:** en el primer arranque el ESP32 crea el SoftAP `ELATO-DEVICE`. Conectate desde el móvil, elige tu red y guarda. El dispositivo pedirá el token automáticamente.

TIP El error mas comun no es de hardware: es que el `JWT_SECRET_KEY` del frontend y del servidor edge no coinciden, o que la MAC no esta dada de alta.

Setup rapido sin desplegar nada (ELATO_MODE)

- En `Config.h` deja `#define ELATO_MODE` (ya viene por defecto).
- Compila y flashea como arriba.
- Conecta el WiFi por el SoftAP `ELATO-DEVICE`.
- Registra el dispositivo en **www.elatoai.com** con su MAC y vinculalo a tu cuenta/agente. Los certificados ya estan incluidos en `Config.cpp`.

Personalizacion

- **Nombre del portal WiFi:** `firmware-arduino/src/main.cpp`, llamada a `WifiManager.startBackgroundTask("ELATO-DEVICE")`.
- **URLs y puertos:** `firmware-arduino/src/Config.cpp` (bloques por modo).
- **Certificados CA:** `Config.cpp`, variables `CA_cert` y `Vercel_CA_cert`.

ADVERTENCIA No publiques tu `JWT_SECRET_KEY` ni claves de IA en un repositorio. Mantenlas en archivos `.env` / variables de entorno, nunca en código commitado.

Solucion de problemas

Sintoma	Causa probable	Solucion
El dispositivo nunca conecta al WebSocket	La MAC no esta registrada en la tabla <code>devices</code>	Da de alta la MAC y vincula el <code>user_code</code> , o pon <code>SKIP_DEVICE_REGISTRATION=True</code> para probar
Respuesta <code>"User not found"</code> en el monitor	<code>getUserByMacAddress</code> no encuentra la MAC	Verifica que la MAC en <code>devices</code> coincide exactamente con la del ESP32
Obtiene token pero el WSS se rechaza/ desconecta	<code>JWT_SECRET_KEY</code> distinto entre frontend y servidor edge	Usa la misma clave en ambos <code>.env</code>
Fallo de handshake SSL / no conecta por HTTPS	Certificado <code>CA_cert</code> / <code>Vercel_CA_cert</code> incorrecto o caducado	Pega el certificado CA correcto del servidor en <code>Config.cpp</code>
No aparece el portal <code>ELATO-DEVICE</code>	Ya hay un WiFi guardado, o no arranco el SoftAP	Haz un factory reset; revisa <code>fallbackToSoftAp(true)</code> en <code>main.cpp</code>
No se ve nada en el monitor serie del Zero	Falta USB-CDC nativo (el Zero no tiene chip UART)	Confirma <code>ARDUINO_USB_CDC_ON_BOOT=1</code> en <code>platformio.ini</code>
Cambie de cuenta/ agente y sigue usando el viejo	El token antiguo persiste en NVS	Haz factory reset para borrar <code>auth_token</code> y volver a pedirlo
En DEV_MODE no conecta	IP local incorrecta o servidor no levantado	Ajusta la IP en <code>Config.cpp</code> (DEV) y arranca el servidor edge local

Referencia rapida

```
SELECCION DE MODO (firmware-arduino/src/Config.h)
#define DEV_MODE      -> servidores locales, sin SSL
#define PROD_MODE     -> tus servidores + tus certificados
#define ELATO_MODE    -> nube de Elato (por defecto)
#define VOICE_SERVER_DENO      -> ws_path "/"
#define VOICE_SERVER_CLOUDFLARE -> ws_path "/ws/esp32"

REGISTRO DEL DISPOSITIVO (frontend / Supabase)
1. Insertar MAC + user_code en tabla "devices"
2. Vincular user_code en Settings (webapp)
3. SKIP_DEVICE_REGISTRATION=False en produccion

CREDENCIALES CLAVE (.env)
JWT_SECRET_KEY      -> IGUAL en frontend y servidor edge
SUPABASE_URL / ANON -> proyecto Supabase
OPENAI_API_KEY / ... -> proveedor de IA (solo servidor/frontend)

FLUJO DEL TOKEN (firmware)
WiFi -> isDeviceRegistered()
      -> GET /api/generate_auth_token?macAddress=MAC
      -> token JWT -> NVS ("auth"/"auth_token")
      -> websocketSetup() "Authorization: Bearer <token>"

COMANDOS
pio run -t upload      # flashear firmware
pio run -t uploadfs    # subir startup.mp3 a SPIFFS
pio device monitor     # ver MAC y logs (USB-CDC)

SoftAP de configuracion WiFi: ELATO-DEVICE
```